

Chapter 1. LLM Fundamentals with LangChain

The [Preface](#) gave you a taste of the power of LLM prompting, where we saw firsthand the impact that different prompting techniques can have on what you get out of LLMs, especially when judiciously combined. The challenge in building good LLM applications is, in fact, in how to effectively construct the prompt sent to the model and process the model's prediction to return an accurate output (see [Figure 1-1](#)).

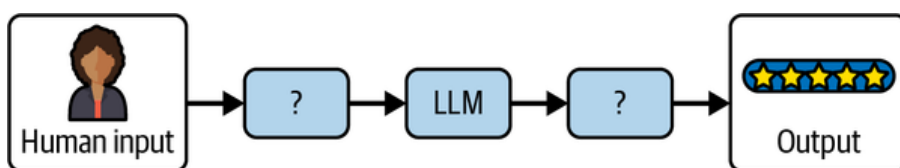


Figure 1-1. The challenge in making LLMs a useful part of your application

If you can solve this problem, you are well on your way to building LLM applications, simple and complex alike. In this chapter, you'll learn more about how LangChain's building blocks map to LLM concepts and how, when combined effectively, they enable you to build LLM applications. But first, the sidebar [“Why LangChain?”](#) is a brief primer on why we think it useful to use LangChain to build LLM applications.

Other formats available



Audio Summary



You can of course build LLM applications without LangChain. The most obvious alternative is to use the software development kit (SDK)—the package exposing the methods of their HTTP API as functions in the programming language of your choice—of the LLM provider you tried first (for example, OpenAI). We think learning LangChain will pay off in the short term and over the long run because of the following factors:

Prebuilt common patterns

LangChain comes with reference implementations of the most common LLM application patterns (we mentioned some of these in the [Preface](#): chain-of-thought, tool calling, and others). This is the quickest way to get started with LLMs and might often be all you need. We'd suggest starting any new application from these and checking whether the results out of the box are good enough for your use case. If not, then see the next item for the other half of the LangChain libraries.

Interchangeable building blocks

These are components that can be easily swapped out for alternatives. Every component (an LLM, chat model, output parser, and so on—more on these shortly) follows a shared specification, which makes your application future-proof. As new capabilities are released by model providers and as your needs change, you can evolve your application without rewriting it each time.

Throughout this book we make use of the following major components in the code examples:

- LLM/chat model: OpenAI
- Embeddings: OpenAI
- Vector store: PGVector

You can swap out each of these for any of the alternatives listed on the following pages:

Chat models

See the [LangChain documentation](#). If you don't want to use OpenAI (a commercial API) we suggest [Anthropic](#) as a commercial alternative or [Ollama](#) as an open source one.

Embeddings

See the [LangChain documentation](#). If you (a commercial API) we suggest [Cohere](#) as or [Ollama](#) as an open source one.

Other formats available

🔊 Audio Summary 

Vector stores

See the [LangChain documentation](#). If you don't want to use PGVector (an open source extension to the popular SQL database Postgres) we suggest using either [Weaviate](#) (a dedicated vector store) or [OpenSearch](#) (vector search features that are part of a popular search database).

This effort goes beyond, for instance, all LLMs having the same methods, with similar arguments and return values. Let's look at the example of chat models and two popular LLM providers, OpenAI and Anthropic. Both have a chat API which receives *chat messages* (loosely defined as objects with a type string and a content string) and returns a new message generated by the model. But if you try to use both models in the same conversation, you'll immediately run into issues, as their chat message formats are subtly incompatible. LangChain abstracts away these differences to enable building applications that are truly independent of a particular provider. For instance, with LangChain, a chatbot conversation where you use both OpenAI and Anthropic models works.

Finally, as you build out your LLM applications with several of these components, we've found it useful to have the *orchestration* capabilities of LangChain:

- All major components are instrumented by the callbacks system for observability (more on this in [Chapter 8](#)).
- All major components implement the same interface (more on this toward the end of this chapter).
- Long-running LLM applications can be interrupted, resumed, or re-tried (more on this in [Chapter 6](#)).

Getting Set Up with LangChain

To follow along with the rest of the chapter, and the chapters to come, we recommend setting up LangChain on your computer first.

See the instructions in the [Preface](#) regarding setting up an OpenAI account and complete these if you haven't yet. If you prefer using a different LLM provider, see [“Why LangChain?”](#) for alternatives.

Then head over to the [API Keys page](#) on the OpenAI website (after logging in to your OpenAI account), create an API key, and save it—you'll need it soon.

NOTE

In this book, we'll show code examples in both Python and JavaScript. LangChain offers the same functionality in both languages, so just pick the one you're most comfortable with and follow the respective code snippets throughout the book (the code examples for each language are equivalent).

Other formats available

 [Audio Summary](#) 

First, some setup instructions for readers using Python:

1. Ensure that you have Python installed. See the [instructions for your operating system](#).
2. Install Jupyter if you want to run the examples in a notebook environment. You can do this by running `pip install notebook` in your terminal.
3. Install the LangChain library by running the following commands in your terminal:

```
pip install langchain langchain-openai langchain-community  
pip install langchain-text-splitters langchain-postgres
```

4. Take the OpenAI API key you generated at the beginning of this section and make it available in your terminal environment. You can do this by running the following:

```
export OPENAI_API_KEY=your-key
```

5. Don't forget to replace `your-key` with the API key you generated previously.
6. Open a Jupyter notebook by running this command:

```
jupyter notebook
```

You're now ready to follow along with the Python code examples.

Here are the instructions for readers using JavaScript:

1. Take the OpenAI API key you generated at the beginning of this section and make it available in your terminal environment. You can do this by running the following:

```
export OPENAI_API_KEY=your-key
```

2. Don't forget to replace `your-key` with the API key you generated previously.
3. If you want to run the examples as Node.js scripts, install Node by following the [instructions](#).
4. Install the LangChain libraries by running the following commands in your terminal:

```
npm install langchain @langchain/  
npm install @langchain/core pg
```

Other formats available

🔊 Audio Summary 🔗

5. Take each example, save it as a `.js` file and run it with `node ./file.js`.

Using LLMs in LangChain

To recap, LLMs are the driving engine behind most generative AI applications. LangChain provides two simple interfaces to interact with any LLM API provider:

- Chat models
- LLMs

The LLM interface simply takes a string prompt as input, sends the input to the model provider, and then returns the model prediction as output.

Let's import LangChain's OpenAI LLM wrapper to `invoke` a model prediction using a simple prompt:

Python

```
from langchain_openai.llms import OpenAI

model = OpenAI(model="gpt-3.5-turbo")

model.invoke("The sky is")
```

JavaScript

```
import { OpenAI } from "@langchain/openai";

const model = new OpenAI({ model: "gpt-3.5-turbo" });

await model.invoke("The sky is");
```

The output:

Blue!

Other formats available

 Audio Summary 

TIP

Notice the parameter `model` passed to `OpenAI`. This is the most common parameter to configure when using an LLM or chat model, the underlying model to use, as most providers offer several models with different trade-offs in capability and cost (usually larger models are more capable, but also more expensive and slower). See [OpenAI's overview](#) of the models they offer.

Other useful parameters to configure include the following, offered by most providers.

`temperature`

This controls the sampling algorithm used to generate output. Lower values produce more predictable outputs (for example, 0.1), while higher values generate more creative, or unexpected, results (such as 0.9). Different tasks will need different values for this parameter. For instance, producing structured output usually benefits from a lower temperature, whereas creative writing tasks do better with a higher value:

`max_tokens`

This limits the size (and cost) of the output. A lower value may cause the LLM to stop generating the output before getting to a natural end, so it may appear to have been truncated.

Beyond these, each provider exposes a different set of parameters. We recommend looking at the documentation for the one you choose. For an example, refer to [OpenAI's platform](#).

Alternatively, the chat model interface enables back and forth conversations between the user and model. The reason why it's a separate interface is because popular LLM providers like OpenAI differentiate messages sent to and from the model into *user*, *assistant*, and *system* roles (here *role* denotes the type of content the message contains):

System role

Used for instructions the model should use to answer a user question

User role

Used for the user's query and any other content produced by the user

Assistant role

Used for content generated by the model

The chat model's interface makes it easier to connect different versions in your AI chatbot application. Here's an example utilizing LangChain's ChatOpenAI model:

Python

Other formats available

🔊 Audio Summary 🔗

```

from langchain_openai.chat_models import ChatOpenAI
from langchain_core.messages import HumanMessage

model = ChatOpenAI()
prompt = [HumanMessage("What is the capital of France?")]

model.invoke(prompt)

```

JavaScript

```

import { ChatOpenAI } from '@langchain/openai'
import { HumanMessage } from '@langchain/core/messages'

const model = new ChatOpenAI()
const prompt = [new HumanMessage('What is the capital of France?')]

await model.invoke(prompt)

```

The output:

```

AIMessage(content='The capital of France is Paris.')

```

Instead of a single prompt string, chat models make use of different types of chat message interfaces associated with each role mentioned previously. These include the following:

HumanMessage

A message sent from the perspective of the human, with the user role

AIMessage

A message sent from the perspective of the AI that the human is interacting with, with the assistant role

SystemMessage

A message setting the instructions the AI should follow, with the system role

ChatMessage

A message allowing for arbitrary setting of role

Let's incorporate a `SystemMessage` instruction

Python

```

from langchain_core.messages import HumanMessage, SystemMessage
from langchain_openai.chat_models import ChatOpenAI

```

Other formats available

 Audio Summary 

```

model = ChatOpenAI()
system_msg = SystemMessage(
    '''You are a helpful assistant that responds to questions with three
       exclamation marks.'''
)
human_msg = HumanMessage('What is the capital of France?')

model.invoke([system_msg, human_msg])

```

JavaScript

```

import { ChatOpenAI } from "@langchain/openai";
import { HumanMessage, SystemMessage } from "@langchain/core/messages";

const model = new ChatOpenAI();
const prompt = [
    new SystemMessage(
        `You are a helpful assistant that responds to questions with three
         exclamation marks.`
    ),
    new HumanMessage("What is the capital of France?"),
];

await model.invoke(prompt);

```

The output:

```
AIMessage('Paris!!!')
```

As you can see, the model obeyed the instruction provided in the `SystemMessage` even though it wasn't present in the user's question. This enables you to preconfigure your AI application to respond in a relatively predictable manner based on the user's input.

Making LLM Prompts Reusable

The previous section showed how the `prompt` instruction significantly influences the model's output. Prompts help the model understand context and generate relevant answers to queries.

Here is an example of a detailed prompt:

Answer the question based on the context provided. Do not answer questions that cannot be answered using the information provided.

Context: The most recent advancements in the field of Natural Language Models (LLMs). These models outperform their smaller counterparts and have become invaluable for developers who are creating applications with NLP capabilities. Developers can tap into these models through Hugging Face's `transformers` library, or by utilizing OpenAI and Cohere's offerings through

Other formats available

 Audio Summary 

the ``openai`` and ``cohere`` libraries, respectively.

Question: Which model providers offer LLMs?

Answer:

Although the prompt looks like a simple string, the challenge is figuring out what the text should contain and how it should vary based on the user's input. In this example, the Context and Question values are hard-coded, but what if we wanted to pass these in dynamically?

Fortunately, LangChain provides prompt template interfaces that make it easy to construct prompts with dynamic inputs:

Python

```
from langchain_core.prompts import PromptTemplate

template = PromptTemplate.from_template("""Answer the question based on the
context below. If the question cannot be answered using the information
provided, answer with "I don't know".

Context: {context}

Question: {question}

Answer: """)

template.invoke({
    "context": """The most recent advancements in NLP are being driven by Large
Language Models (LLMs). These models outperform their smaller
counterparts and have become invaluable for developers who are creating
applications with NLP capabilities. Developers can tap into these
models through Hugging Face's `transformers` library, or by utilizing
OpenAI and Cohere's offerings through the `openai` and `cohere`
libraries, respectively.""",
    "question": "Which model providers offer LLMs?"
})
```

JavaScript

```
import { PromptTemplate } from '@langchain/core/prompts'

const template = PromptTemplate.fromTemplate(`Answer the question based on the
context below. If the question cannot be answered using the information
provided, answer with "I don't know".

Context: {context}

Question: {question}

Answer: `)
```

Other formats available

 Audio Summary 

```

await template.invoke({
    context: `The most recent advancements in NLP are being driven by Large
    Language Models (LLMs). These models outperform their smaller
    counterparts and have become invaluable for developers who are creating
    applications with NLP capabilities. Developers can tap into these models
    through Hugging Face's ``transformers`` library, or by utilizing OpenAI
    and Cohere's offerings through the ``openai`` and ``cohere`` libraries,
    respectively.``,
    question: "Which model providers offer LLMs?"
})

```

The output:

```

StringPromptValue(text='Answer the question based on the context below. If the
question cannot be answered using the information provided, answer with "I
don\'t know".\n\nContext: The most recent advancements in NLP are being
driven by Large Language Models (LLMs). These models outperform their
smaller counterparts and have become invaluable for developers who are
creating applications with NLP capabilities. Developers can tap into these
models through Hugging Face\'s `transformers` library, or by utilizing
OpenAI and Cohere\'s offerings through the `openai` and `cohere` libraries
respectively.\n\nQuestion: Which model providers offer LLMs?\n\nAnswer: ')

```

This example takes the static prompt from the previous block and makes it dynamic. The `template` contains the structure of the final prompt alongside the definition of where the dynamic inputs will be inserted.

As such, the template can be used as a recipe to build multiple static, specific prompts. When you format the prompt with some specific values—in this case, `context` and `question`—you get a static prompt ready to be passed in to an LLM.

As you can see, the `question` argument is passed dynamically via the `invoke` function. By default, LangChain prompts follow Python’s `f-string` syntax for defining dynamic parameters—any word surrounded by curly braces, such as `{question}`, are placeholders for values passed in at runtime. In the previous example, `{question}` was replaced by “Which model providers offer LLMs?”

Let’s see how we’d feed this into an LLM OpenAI model using LangChain:

Python

```

from langchain_openai.llms import OpenAI
from langchain_core.prompts import PromptTemplate

# both `template` and `model` can be passed to the `invoke` method

template = PromptTemplate.from_template("""Answer the question based on the
context below. If the question cannot be answered using the information
provided, answer with "I don't know".

```

Other formats available

 Audio Summary 

```

Context: {context}

Question: {question}

Answer: ""

model = OpenAI()

# `prompt` and `completion` are the results of using template and model once

prompt = template.invoke({
    "context": ""The most recent advancements in NLP are being driven by Large
    Language Models (LLMs). These models outperform their smaller
    counterparts and have become invaluable for developers who are creating
    applications with NLP capabilities. Developers can tap into these
    models through Hugging Face's `transformers` library, or by utilizing
    OpenAI and Cohere's offerings through the `openai` and `cohere`
    libraries, respectively.""",
    "question": "Which model providers offer LLMs?"
})

completion = model.invoke(prompt)

```

JavaScript

```

import { PromptTemplate } from '@langchain/core/prompts'
import { OpenAI } from '@langchain/openai'

const model = new OpenAI()
const template = PromptTemplate.fromTemplate(`Answer the question based on the
context below. If the question cannot be answered using the information
provided, answer with "I don't know".

Context: {context}

Question: {question}

Answer: `)

const prompt = await template.invoke({
    context: `The most recent advancements in NLP are being driven by Large
    Language Models (LLMs). These models outperform their smaller
    counterparts and have become invaluable for developers who are creating
    applications with NLP capabilities. Developers can tap into these models
    through Hugging Face's ``transformers`` library, or by utilizing OpenAI
    and Cohere's offerings through the ``openai`` and ``cohere`` libraries,
    respectively.`,
    question: "Which model providers offer LLMs?"
})

await model.invoke(prompt)

```

Other formats available

 Audio Summary 

The output:

Hugging Face's `transformers` library, OpenAI using the `openai` library, and Cohere using the `cohere` library offer LLMs.

If you're looking to build an AI chat application, the `ChatPromptTemplate` can be used instead to provide dynamic inputs based on the role of the chat message:

Python

```
from langchain_core.prompts import ChatPromptTemplate
template = ChatPromptTemplate.from_messages([
    ('system', '''Answer the question based on the context below. If the
        question cannot be answered using the information provided, answer with
        "I don't know."'''),
    ('human', 'Context: {context}'),
    ('human', 'Question: {question}'),
])

template.invoke({
    "context": """The most recent advancements in NLP are being driven by Large
        Language Models (LLMs). These models outperform their smaller
        counterparts and have become invaluable for developers who are creating
        applications with NLP capabilities. Developers can tap into these
        models through Hugging Face's `transformers` library, or by utilizing
        OpenAI and Cohere's offerings through the `openai` and `cohere`
        libraries, respectively."""
    "question": "Which model providers offer LLMs?"
})
```

JavaScript

```
import { ChatPromptTemplate } from '@langchain/core/prompts'

const template = ChatPromptTemplate.fromMessages([
    ['system', `Answer the question based on the context below. If the question
        cannot be answered using the information provided, answer with "I
        don't know."`],
    ['human', 'Context: {context}'],
    ['human', 'Question: {question}'],
])

await template.invoke({
    context: `The most recent advancements in NLP are being driven by Large
        Language Models (LLMs). These models outperform their smaller
        counterparts and have become invaluable for developers who are creating
        applications with NLP capabilities through Hugging Face's `transformers`
        and Cohere's offerings through the `openai` and `cohere` libraries,
        respectively.`
    question: "Which model providers offer LLMs?"
})
```

Other formats available

🔊 Audio Summary 🔗

✕ models
OpenAI
aries,

The output:

```
ChatPromptValue(messages=[SystemMessage(content='Answer the question based on the context below. If the question cannot be answered using the information provided, answer with "I don\'t know".'), HumanMessage(content="Context: The most recent advancements in NLP are being driven by Large Language Models (LLMs). These models outperform their smaller counterparts and have become invaluable for developers who are creating applications with NLP capabilities. Developers can tap into these models through Hugging Face\'s `transformers` library, or by utilizing OpenAI and Cohere\'s offerings through the `openai` and `cohere` libraries, respectively."), HumanMessage(content='Question: Which model providers offer LLMs?')])
```

Notice how the prompt contains instructions in a `SystemMessage` and two instances of `HumanMessage` that contain dynamic `context` and `question` variables. You can still format the template in the same way and get back a static prompt that you can pass to a large language model for a prediction output:

Python

```
from langchain_openai.chat_models import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate

# both `template` and `model` can be reused many times

template = ChatPromptTemplate.from_messages([
    ('system', '''Answer the question based on the context below. If the question cannot be answered using the information provided, answer with "I don\'t know".'''),
    ('human', 'Context: {context}'),
    ('human', 'Question: {question}'),
])

model = ChatOpenAI()

# `prompt` and `completion` are the results of using template and model once

prompt = template.invoke({
    "context": """The most recent advancements in NLP are being driven by Large Language Models (LLMs). These models outperform their smaller counterparts and have become invaluable for developers who are creating applications with NLP capabilities. Developers can tap into these models through Hugging Face's `transformers` library, or by utilizing OpenAI and Cohere's offerings through the `openai` and `cohere` libraries, respectively."""
    "question": "Which model providers offer LLMs?"
})

model.invoke(prompt)
```

Other formats available

 Audio Summary 

JavaScript

```
import { ChatPromptTemplate } from '@langchain/core/prompts'
import { ChatOpenAI } from '@langchain/openai'

const model = new ChatOpenAI()
const template = ChatPromptTemplate.fromMessages([
  ['system', `Answer the question based on the context below. If the question
    cannot be answered using the information provided, answer with "I
    don't know".`],
  ['human', 'Context: {context}'],
  ['human', 'Question: {question}'],
])

const prompt = await template.invoke({
  context: `The most recent advancements in NLP are being driven by Large
    Language Models (LLMs). These models outperform their smaller
    counterparts and have become invaluable for developers who are creating
    applications with NLP capabilities. Developers can tap into these models
    through Hugging Face's ``transformers`` library, or by utilizing OpenAI
    and Cohere's offerings through the ``openai`` and ``cohere`` libraries,
    respectively.`,
  question: "Which model providers offer LLMs?"
})

await model.invoke(prompt)
```

The output:

```
AIMessage(content="Hugging Face's ``transformers`` library, OpenAI using the
  ``openai`` library, and Cohere using the ``cohere`` library offer LLMs.")
```

Getting Specific Formats out of LLMs

Plain text outputs are useful, but there may be use cases where you need the LLM to generate a *structured* output—that is, output in a machine-readable format, such as JSON, XML, CSV, or even in a programming language such as Python or JavaScript. This is very useful when you intend to hand that output off to some other piece of code, making an LLM play a part in your larger application.

JSON Output

The most common format to generate with LLMs is JSON. JSON outputs can (for example) be sent over the wire to your frontend code or be saved to a database.

When generating JSON, the first task is to define a schema for the LLM to respect when producing the output. The schema is defined in the prompt, along with the text you want to use as the source. Let's see an example:

Python

Other formats available

🔊 Audio Summary 🔗

```

from langchain_openai import ChatOpenAI
from langchain_core.pydantic_v1 import BaseModel

class AnswerWithJustification(BaseModel):
    '''An answer to the user's question along with justification for the
    answer.'''
    answer: str
    '''The answer to the user's question'''
    justification: str
    '''Justification for the answer'''

llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0)
structured_llm = llm.with_structured_output(AnswerWithJustification)

structured_llm.invoke("""What weighs more, a pound of bricks or a pound
of feathers""")

```

JavaScript

```

import { ChatOpenAI } from '@langchain/openai'
import { z } from "zod";

const answerSchema = z
  .object({
    answer: z.string().describe("The answer to the user's question"),
    justification: z.string().describe(`Justification for the
    answer`),
  })
  .describe(`An answer to the user's question along with justification for
  the answer.`);

const model = new ChatOpenAI({
  model: "gpt-3.5-turbo",
  temperature: 0,
}).withStructuredOutput(answerSchema)
await model.invoke("What weighs more, a pound of bricks or a pound of feathers")

```

The output:

```

{
  answer: "They weigh the same",
  justification: "Both a pound of bricks and a pound of feathers weigh one pou
  The weight is the same, but the volu"... 42 more characters
}

```

So, first define a schema. In Python, this is easier with the `pydantic` library used for validating data against schemas. In JavaScript, you can do with Zod (an equivalent library). The method `structured_llm.with_structured_output` will use that schema for two things:

- The schema will be converted to a `JSONSchema` object (a JSON format used to describe the shape [types, names, descriptions] of JSON data),

Other formats available

 Audio Summary 

which will be sent to the LLM. For each LLM, LangChain picks the best method to do this, usually function calling or prompting.

- The schema will also be used to validate the output returned by the LLM before returning it; this ensures the output produced respects the schema you passed in exactly.

Other Machine-Readable Formats with Output Parsers

You can also use an LLM or chat model to produce output in other formats, such as CSV or XML. This is where output parsers come in handy. *Output parsers* are classes that help you structure large language model responses. They serve two functions:

Providing format instructions

Output parsers can be used to inject some additional instructions in the prompt that will help guide the LLM to output text in the format it knows how to parse.

Validating and parsing output

The main function is to take the textual output of the LLM or chat model and render it to a more structured format, such as a list, XML, or other format. This can include removing extraneous information, correcting incomplete output, and validating the parsed values.

Here's an example of how an output parser works:

Python

```
from langchain_core.output_parsers import CommaSeparatedListOutputParser
parser = CommaSeparatedListOutputParser()
items = parser.invoke("apple, banana, cherry")
```

JavaScript

```
import { CommaSeparatedListOutputParser } from '@langchain/core/output_parsers'

const parser = new CommaSeparatedListOutputParser()

await parser.invoke("apple, banana, cherry")
```

The output:

```
['apple', 'banana', 'cherry']
```

Other formats available

🔊 Audio Summary 🔗

LangChain provides a variety of output parsers for various use cases, including CSV, XML, and more. We'll see how to combine output parsers with models and prompts in the next section.

Assembling the Many Pieces of an LLM Application

The key components you've learned about so far are essential building blocks of the LangChain framework. Which brings us to the critical question: How do you combine them effectively to build your LLM application?

Using the Runnable Interface

As you may have noticed, all the code examples used so far utilize a similar interface and the `invoke()` method to generate outputs from the model (or prompt template, or output parser). All components have the following:

- There is a common interface with these methods:
 - `invoke`: transforms a single input into an output
 - `batch`: efficiently transforms multiple inputs into multiple outputs
 - `stream`: streams output from a single input as it's produced
- There are built-in utilities for retries, fallbacks, schemas, and runtime configurability.
- In Python, each of the three methods have `asyncio` equivalents.

As such, all components behave the same way, and the interface learned for one of them applies to all:

Python

```
from langchain_openai.llms import ChatOpenAI

model = ChatOpenAI()

completion = model.invoke('Hi there!')
# Hi!

completions = model.batch(['Hi there!', 'Bye!'])
# ['Hi!', 'See you!']

for token in model.stream('Bye!'):
    print(token)
# Good
# bye
# !
```

Other formats available

🔊 Audio Summary 🔗

```
import { ChatOpenAI } from '@langchain/openai'

const model = new ChatOpenAI()

const completion = await model.invoke('Hi there!')
// Hi!

const completions = await model.batch(['Hi there!', 'Bye!'])
// ['Hi!', 'See you!']

for await (const token of await model.stream('Bye!')) {
  console.log(token)
  // Good
  // bye
  // !
}
```

In this example, you see how the three main methods work:

- `invoke()` takes a single input and returns a single output.
- `batch()` takes a list of outputs and returns a list of outputs.
- `stream()` takes a single input and returns an iterator of parts of the output as they become available.

In some cases, where the underlying component doesn't support iterative output, there will be a single part containing all output.

You can combine these components in two ways:

Imperative

Call your components directly, for example, with `model.invoke(...)`

Declarative

Use LangChain Expression Language (LCEL), as covered in an upcoming section

[Table 1-1](#) summarizes their differences, and we'll see each in action next.

Table 1-1. The main differences between imperative and declarative composition.

	Imperative	Declarative
Syntax	All of Python or JavaScript	
Parallel execution	Python: with threads or coroutines JavaScript: with <code>Promise.all()</code>	
Streaming	With <code>yield</code> keyword	Automatic
Async execution	With <code>async</code> functions	Automatic

Other formats available

Audio Summary

Imperative Composition

Imperative composition is just a fancy name for writing the code you're used to writing, composing these components into functions and classes. Here's an example combining prompts, models, and output parsers:

Python

```
from langchain_openai.chat_models import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.runnables import chain

# the building blocks

template = ChatPromptTemplate.from_messages([
    ('system', 'You are a helpful assistant.'),
    ('human', '{question}'),
])

model = ChatOpenAI()

# combine them in a function
# @chain decorator adds the same Runnable interface for any function you write

@chain
def chatbot(values):
    prompt = template.invoke(values)
    return model.invoke(prompt)

# use it

chatbot.invoke({"question": "Which model providers offer LLMs?"})
```

JavaScript

```
import {ChatOpenAI} from '@langchain/openai'
import {ChatPromptTemplate} from '@langchain/core/prompts'
import {RunnableLambda} from '@langchain/core/runnables'

// the building blocks

const template = ChatPromptTemplate.fromMessages([
  ['system', 'You are a helpful assistant.'],
  ['human', '{question}'],
])

const model = new ChatOpenAI()

// combine them in a function
// RunnableLambda adds the same Runnable interface for any function you write

const chatbot = RunnableLambda.from(async values => {
  const prompt = await template.invoke(values)
  return await model.invoke(prompt)
})
```

Other formats available

🔊 Audio Summary 🔗

```

    })

    // use it

    await chatbot.invoke({
      "question": "Which model providers offer LLMs?"
    })

```

The output:

```

AIMessage(content="Hugging Face's `transformers` library, OpenAI using the
`openai` library, and Cohere using the `cohere` library offer LLMs.")

```

The preceding is a complete example of a chatbot, using a prompt and chat model. As you can see, it uses familiar Python syntax and supports any custom logic you might want to add in that function.

On the other hand, if you want to enable streaming or async support, you'd have to modify your function to support it. For example, streaming support can be added as follows:

Python

```

@chain
def chatbot(values):
    prompt = template.invoke(values)
    for token in model.stream(prompt):
        yield token

for part in chatbot.stream({
    "question": "Which model providers offer LLMs?"
}):
    print(part)

```

JavaScript

```

const chatbot = RunnableLambda.from(async function* (values) {
  const prompt = await template.invoke(values)
  for await (const token of await model.stream(prompt)) {
    yield token
  }
})

for await (const token of await chatbot.invoke({
  "question": "Which model provider offer LLMs?"
})) {
  console.log(token)
}

```

Other formats available

🔊 Audio Summary 🔗

The output:

```

AIMessageChunk(content="Hugging")
AIMessageChunk(content=" Face's")
AIMessageChunk(content=" `transformers`")
...

```

So, either in JS or Python, you can enable streaming for your custom function by yielding the values you want to stream and then calling it with `stream`.

For asynchronous execution, you'd rewrite your function like this:

Python

```

@chain
async def chatbot(values):
    prompt = await template.ainvoke(values)
    return await model.ainvoke(prompt)

await chatbot.ainvoke({"question": "Which model providers offer LLMs?"})
# > AIMessage(content="""Hugging Face's `transformers` library, OpenAI using
    the `openai` library, and Cohere using the `cohere` library offer LLMs.""")

```

This one applies to Python only, as asynchronous execution is the only option in JavaScript.

Declarative Composition

LCEL is a *declarative language* for composing LangChain components. LangChain compiles LCEL compositions to an *optimized execution plan*, with automatic parallelization, streaming, tracing, and async support.

Let's see the same example using LCEL:

Python

```

from langchain_openai.chat_models import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate

# the building blocks

template = ChatPromptTemplate.from_messages([
    ('system', 'You are a helpful assistant.'),
    ('human', '{question}'),
])

model = ChatOpenAI()

# combine them with the | operator

chatbot = template | model

# use it

```

Other formats available

🔊 Audio Summary 🔗

```
chatbot.invoke({"question": "Which model providers offer LLMs?"})
```

JavaScript

```
import { ChatOpenAI } from '@langchain/openai'
import { ChatPromptTemplate } from '@langchain/core/prompts'
import { RunnableLambda } from '@langchain/core/runnables'

// the building blocks

const template = ChatPromptTemplate.fromMessages([
  ['system', 'You are a helpful assistant.'],
  ['human', '{question}'],
])

const model = new ChatOpenAI()

// combine them in a function

const chatbot = template.pipe(model)

// use it

await chatbot.invoke({
  "question": "Which model providers offer LLMs?"
})
```

The output:

```
AIMessage(content="Hugging Face's `transformers` library, OpenAI using the  
`openai` library, and Cohere using the `cohere` library offer LLMs.")
```

Crucially, the last line is the same between the two examples—that is, you use the function and the LCEL sequence in the same way, with `invoke/stream/batch`. And in this version, you don't need to do anything else to use streaming:

Python

```
chatbot = template | model

for part in chatbot.stream({
  "question": "Which model providers offer LLMs?"
}):
    print(part)
    # > AIMessageChunk(content="Hug
    # > AIMessageChunk(content=" Fa
    # > AIMessageChunk(content=" `transformers`)
    # ...
```

Other formats available

🔊 Audio Summary 🔗

```
const chatbot = template.pipe(model)

for await (const token of await chatbot.stream({
  "question": "Which model providers offer LLMs?"
})) {
  console.log(token)
}
```

And, for Python only, it's the same for using asynchronous methods:

Python

```
chatbot = template | model

await chatbot.ainvoke({
  "question": "Which model providers offer LLMs?"
})
```

Summary

In this chapter, you've learned about the building blocks and key components necessary to build LLM applications using LangChain. LLM applications are essentially a chain consisting of the large language model to make predictions, the prompt instruction(s) to guide the model toward a desired output, and an optional output parser to transform the format of the model's output.

All LangChain components share the same interface with `invoke`, `stream`, and `batch` methods to handle various inputs and outputs. They can either be combined and executed imperatively by calling them directly or declaratively using LCEL.

The imperative approach is useful if you intend to write a lot of custom logic, whereas the declarative approach is useful for simply assembling existing components with limited customization.

In [Chapter 2](#), you'll learn how to provide external data to your AI chatbot as *context* so that you can build an LLM application that enables you to “chat” with your data.

Other formats available

 Audio Summary 